

Rapid Notes

Is it a useful app, and could it produce a small passive income on Google Play?

Subject

Rapid Notes v1 — offline-first PWA

Codebase

3,179 lines · single file · 12 commits

Prepared for

LP

The short answer

The app is genuinely good and the passive income is genuinely unlikely. Those two things are both true, and they're true for unrelated reasons.

Rapid Notes has a real idea at its centre — shorthand expansion plus prefix classification in a capture-first dock — and the execution shows craft that most indie note apps don't have. But notes is the most saturated free category on Android, your chosen ad model can't be implemented inside a PWA wrapper, and "light upkeep with no marketing" caps installs in the low hundreds. At that volume the arithmetic doesn't reach meaningful money no matter how good the app is.

\$6 - \$610

Realistic year-1 revenue range

~\$6/mo

Base-case outcome

\$37

Year-1 cash cost to publish

3-5 wks

Time to get live on Play

The rest of this report explains how I got there, what's actually strong in the build, the three things that would stop someone paying for it, the specific Play Store obstacles waiting for you, and — at the end — the two changes that would move the numbers most, plus a cheaper alternative worth considering.

1. What you've actually built

Rapid Notes is a single-file, no-build progressive web app: 136 KB of HTML, CSS and JavaScript in one document, backed by `localStorage` for structured data and IndexedDB for media blobs. It sits on top of a personal design system ("JigsawNote") with its own token layer and component library. Twelve commits across three weeks, solo.

AREA

WHAT'S THERE

Capture

Persistent bottom dock ("Log a thought..."), category prefixes (! to-do, ? question, * idea) that stack, PWA share target, "New note" launcher shortcut

Shorthand engine

User dictionary with folder-scoped entries that inherit to sub-folders; live tokens `{date}` `{time}` `{day}` `{datetime}`; per-folder on/off

Organisation

Nested folder tree, pins, recents, home dashboard, search, select-mode with bulk ops, swipe gestures, desktop drag-and-drop between category sections

Editor	Bold / italic / underline / strike, checklists with Enter-Backspace handling, undo-redo stack, plain-text-only paste, tag chips
Media	Voice recording via MediaRecorder with a full-screen timer overlay; photo attach; blobs in IndexedDB
Safety	Recycle bin with a 15-day TTL; TXT export; print-to-PDF export; file import
Platform polish	Android back-gesture handled as a history stack that peels UI layers; keyboard-aware dock via visualViewport; native pull-to-refresh suppressed; three themes; offline app-shell service worker
AI (optional)	Bring-your-own DeepSeek key — action-item extraction into checklists, and folder summaries cached by a cheap fingerprint

2. Where it's genuinely good

01 The shorthand engine is a real differentiator

Almost no consumer note app ships a user-editable expansion dictionary, and scoping entries to a folder tree — so `rx` means one thing in a work folder and another in a personal one — is a nicer idea than most text expanders manage. Combined with the stackable prefix classifier, you have a coherent thesis: *typing less is the product*. That is a thing a specific kind of person wants badly.

02 The platform details are the ones that actually matter

Back-gesture as a history stack, keyboard-aware dock, plain-text paste, suppressed pull-to-refresh, opaque caching of CDN assets so icons survive offline. These are exactly the details that separate an app that *feels* native from a web page in a box, and they're the details most solo PWAs skip. Someone paid attention.

03 Local-only is a legitimate 2026 selling point

No account, no server, nothing leaves the device. That is a real position in a market where every competitor wants a login, and it's the kind of thing that earns goodwill in privacy-minded communities.

04 Capture speed is a defensible niche

You're not competing with Notion. You're competing with the two seconds between having a thought and losing it. That's a narrower, more winnable frame than "note app."

3. What would stop someone paying for it

These are ranked by how much they hurt. The first one is not a polish item — it's a blocker for charging money.

01 · There is no backup, no sync, and no durable storage — for a notes app

Everything lives in `localStorage` as a single JSON blob rewritten on each save. Clearing site data, reinstalling, or Chrome evicting the origin under storage pressure means **total, silent, unrecoverable data loss**. There's no `navigator.storage.persist()` call to ask the browser not to evict you, and the only escape hatch is a manual TXT export the user has to remember to run.

Separately, `localStorage` caps at roughly 5 MB. A heavy user will hit that wall and `localStorage` will throw an uncaught `QuotaExceededError` mid-save. Media is correctly in IndexedDB; the note text isn't.

You cannot charge for a notes app that can lose all the notes. The first one-star review that says "it deleted everything" ends the listing. Fix this before anything else.

02 · No sync is also the thing people actually pay for

Look at what every paid note app charges for: sync across devices. It's the one feature with obvious, recurring value that justifies a subscription. Rapid Notes deliberately has no backend — which is a fine product decision and a hard commercial one, because it removes the strongest reason to pay. It also means the "subscription" option you flagged has nothing to sell yet.

03 · No widget, no quick tile, no lock-screen entry

For a capture app, the biggest competitive feature is being reachable in under a second. Google Keep has a home-screen widget and a quick-settings tile. A Trusted Web Activity can't provide either — you'd need a native shell. Right now the fastest path in is "unlock, find icon, tap," which is exactly what a capture app is supposed to eliminate.

04 · The DeepSeek key is a power-user feature in a consumer app

Realistically almost nobody will paste an API key into a note app. It's also a Play compliance surface: note text is sent to a third-party AI provider, which must be declared in the Data Safety form and the privacy policy. And commercially it's backwards — the value accrues to DeepSeek, not you. It's a great personal feature; it isn't a monetisation hook.

05 · Smaller technical debts

Formatting runs on `document.execCommand`, which is deprecated and the usual source of cross-device editor bugs. Icons come from a third-party CDN at runtime, so a cold first launch with no network shows no icons in an app that advertises offline. And 3,179 lines in one file is workable now and won't be at 6,000.

4. The Play Store path

Publishing a PWA to Play is officially supported through a Trusted Web Activity — Chrome renders your site inside a real app shell, verified as yours via Digital Asset Links. It works, and PWABuilder or Bubblewrap generates the bundle for you. The obstacles are administrative, not technical.

REQUIREMENT	DETAIL
Public HTTPS hosting	The app currently lives in a local folder. You need a domain plus an <code>assetlinks.json</code> at a fixed path. Cloudflare Pages or Netlify free tier is enough; budget ~\$12/yr for the domain.

Developer account	\$25 one-time, non-refundable. Personal accounts require government ID verification, typically 1–7 days.
12 testers, 14 days	Personal accounts created after 13 Nov 2023 must run a closed test with at least 12 unique opted-in testers, continuously, for 14 days before applying for production. Real Google accounts on real devices — emulators don't count, and if a tester drops out the clock resets. This is the practical hurdle: you need to find twelve actual humans.
Quality gates	Lighthouse PWA score ≥ 80 . Failing Digital Asset Links verification, or returning 404/5xx, is treated as an app crash and can get you delisted.
Build target	Android App Bundle; target API 36 (Android 16) from 31 Aug 2026.
Declarations	Live privacy-policy URL, Data Safety form, IARC content rating, target-audience and ads declarations.
Review	Production-access questionnaire ~7 days, then app review up to 7 days.
Realistic end-to-end	3-5+ weeks

One piece of good news: Play's fees dropped in June 2026. Under \$1M/year you pay a 10% service fee plus a 5% billing fee — about **15% all-in** in the US, UK and EEA, versus the old 30%. On a \$3.99 unlock you keep roughly \$3.39.

Low risk: rejection for "minimum functionality"

Google's spam policy targets thin webview wrappers with no real functionality. Rapid Notes isn't one — it has a genuine feature set, works offline, and does something no other listing does. This is worth knowing about, not worrying about.

5. The ads problem — this one is specific to your plan

You said you'd be happy with tiny ads in the menu bar, removable with a one-time payment. That's a sensible model. **It cannot be built inside a Trusted Web Activity.**

Neither Google ad product works in a PWA wrapper

AdMob — the app ad product — has no supported way to render inside a TWA. The TWA is a full-screen Chrome surface under the browser's control, so you can't overlay a native `AdView`. It's an open, unresolved feature request on Google's own `android-browser-helper` repo, and every workaround (WebView instead of TWA, injecting ads, drawing native views on top) is either unsupported, unreliable, or a policy violation.

AdSense — the web ad product — explicitly forbids it. The program policies state that Google ads "may not be ... integrated into a software application (does not apply to AdMob) of any kind." Putting AdSense in your page and shipping that page as an app is exactly the thing that sentence prohibits.

So the ad tier has two honest routes:

- **Drop ads entirely** and go free-with-a-paid-unlock, or paid upfront. Simplest, and for a note app arguably the better product anyway.

- **Build a thin native Android shell** — a WebView host with the Google Mobile Ads SDK, using the WebView API for Ads. This is legitimate and supported. It also means you're now maintaining an Android project, which is no longer "light upkeep."

Note that the ad revenue in the model below is included only for completeness. Given the above, treat it as the ceiling on a route you'd have to build extra scaffolding to reach — and note that even in the bull case it's worth about \$17 a month.

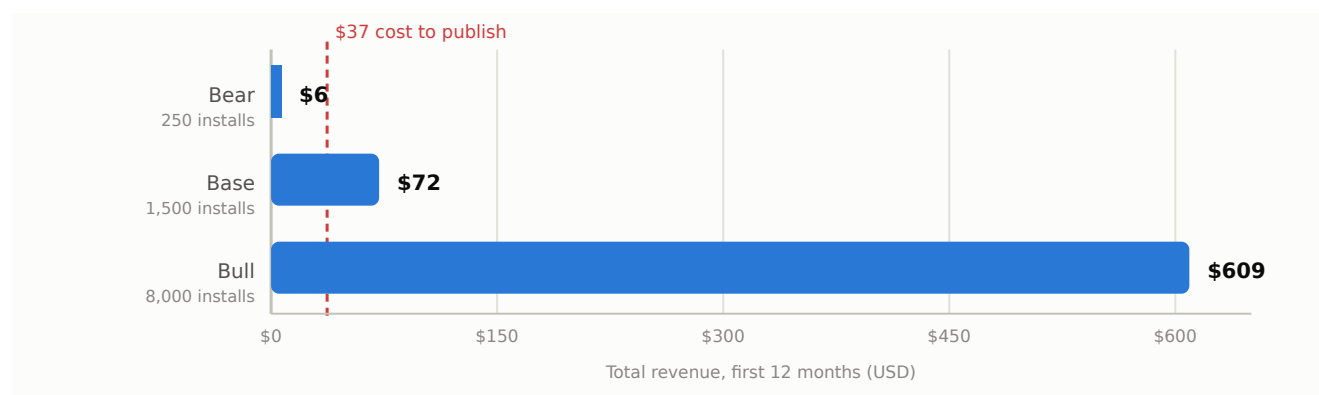
6. The money

Three scenarios, twelve months, built on published benchmarks rather than optimism. The install numbers assume what you described: light upkeep, decent store listing, no sustained marketing.

ASSUMPTION	BEAR	BASE	BULL
Year-1 installs	250	1,500	8,000
Buy the \$3.99 unlock	0.5%	1.0%	1.5%
Paying customers	1	15	120
Unlock revenue, net of Play's 15%	\$4	\$51	\$407
Average monthly actives	8	60	400
Ad revenue (if buildable)	\$2	\$22	\$202
Year-1 revenue	\$6	\$72	\$609
Less year-1 costs (\$25 fee + \$12 domain)	−\$37	−\$37	−\$37
Year-1 profit	−\$31	\$35	\$572
<i>Equivalent per month</i>	<i>\$0.51</i>	<i>\$6.04</i>	<i>\$50.72</i>

Year-1 revenue by scenario

Dashed line marks the \$37 it costs to publish. The bear case never crosses it.



Model assumptions and sources are listed in the appendix. Ad revenue is included at its theoretical ceiling; see section 5.

How the benchmarks anchor this

- **78% of apps released in 2026 have under 1,000 downloads.** The base case of 1,500 already puts you in the better quarter of new listings.
- **Freemium apps convert about 2.1% of downloads to paid by day 35;** hard-paywall apps hit 10.7%. A 1% unlock rate is a deliberately sober read for an optional cosmetic unlock rather than a gate.
- **Median subscription app makes roughly \$72/month one year after launch,** with the top 10% at \$2,500+ and only 4.6% of new apps ever reaching \$10K/month. Note that's *per month* — the base case here reaches \$72 across the entire year, because that median is drawn from apps that market themselves and sell subscriptions.
- **Banner eCPM runs \$0.20-\$0.80 globally,** \$0.50-\$1.50 in tier-1 countries, and productivity apps sit at the low end because sessions are short and users resist interruption.
- **65% of installs come through store search.** With no marketing, your listing's keywords are essentially your entire acquisition channel.

What it would take to hit a real number

Working backwards from the \$3.99 unlock, net of Play's cut:

MONTHLY TARGET	AT 0.5% PAYING	AT 1% PAYING	AT 2% PAYING
\$10 / month	7,100 installs/yr	3,500	1,800
\$50 / month	35,400	17,700	8,800
\$100 / month	70,800	35,400	17,700
\$500 / month	353,800	176,900	88,500

The middle column is the honest one. **Roughly 3,000 installs a year — about 250 a month — to make ten dollars a month.** Around 35,000 a year to make a hundred. Unmarketed apps in saturated categories do not do 35,000 installs a year.

7. The honest verdict

Judged as an app: **useful, well-made, and genuinely differentiated.** If you use it daily and it saves you time, it has already justified itself. The shorthand idea is good enough that a small group of people would like it a lot.

Judged as passive income at light upkeep: **expect under \$10 a month, most likely under \$2, with a meaningful chance of not clearing the \$37 you spend to publish.** That's not a comment on the quality of the build. It's the structure of the market:

- Notes is the most saturated free category on Android, and the default competitor is pre-installed, free, syncs everywhere, and made by the company that owns the store.
- The one feature people reliably pay for — sync — is the one you've architecturally excluded.
- The ad tier you wanted can't be built on the platform you'd ship on.
- "No marketing" in a search-driven store means your ceiling is set by keyword luck.

Passive income from apps is mostly a survivorship story. The apps that earn quietly for years almost all had a non-passive first year.

8. If you want to improve the odds

Ranked by leverage per hour spent.

01 Make the data durable before anything else

Call `navigator.storage.persist()`, move note text out of `localStorage` into IndexedDB, catch quota errors, and add an automatic periodic export — a JSON backup the user can restore, ideally written to their Drive or Downloads folder without being asked. This is a weekend of work and it is the difference between a product you can charge for and one you can't. Do it whether or not you publish.

02 Narrow the positioning until it stops being a note app

"Note app" is unwinnable in store search. "Shorthand capture" isn't. Lead the title, icon and screenshots with the dictionary — it's the thing nobody else has, and it's the thing that gets you found by people already searching for text expanders and abbreviation tools. Your first screenshot should show shorthand expanding, not a list of notes.

03 Pick one monetisation shape and commit

Given section 5, the clean version is: **free app, one-time unlock at \$3.99-\$4.99** for whatever the pro tier is (unlimited dictionary entries, extra themes, media attachments — pick something that doesn't cripple the core loop). Skip ads. Skip the subscription until sync exists to justify it. Play's Digital Goods API handles the purchase inside a TWA and you keep ~85%.

04 Line up your twelve testers before you start

The 14-day clock is fixed and resets if someone drops out. Have twelve committed people with Google accounts ready on day one, or the closed test becomes the reason this project stalls.

05 Decide honestly whether you want sync

Sync is the fork in the road. It's the only credible path to recurring revenue, and it converts this from a weekend project into a service with a backend, a bill, and an obligation. There's no shame in choosing not to — but if you don't, price the app as a one-time purchase and stop thinking about subscriptions.

9. The alternative worth considering

You may not need the Play Store at all.

Rapid Notes is already an installable PWA. Host it on a domain, and anyone on Android can add it to their home screen from Chrome in two taps — same icon, same offline behaviour, same experience the TWA would give them. Then sell a licence key through Gumroad, Lemon Squeezy or Ko-fi.

	PLAY STORE (TWA)	DIRECT PWA + LICENCE
Setup cost	\$25 + domain	Domain only
Time to live	3-5+ weeks	An afternoon
12 testers / 14 days	Required	None
Platform cut	~15%	~5-9%

Review risk	Real	None
Discovery	Store search (65% of installs)	You have to bring everyone
Perceived legitimacy	High	Lower

Discovery is the real trade — and it's a smaller trade than it looks, because at zero marketing the store gives you very little discovery anyway. My suggestion: **ship the PWA route first**. It costs days instead of weeks, and it tells you the one thing the model can't — whether anyone outside your own head wants shorthand capture. If people do, the Play listing is a much better bet with real usage data and a handful of reviews behind it. If they don't, you've saved yourself five weeks and a tester hunt.

Appendix — assumptions and sources

Model assumptions

- Unlock price \$3.99; Play take 10% service fee + 5% billing fee = 15% (sub-\$1M tier, US/UK/EEA, effective 30 June 2026). Net \$3.39 per sale.
- Installs accrue roughly linearly over twelve months, so average monthly actives are taken as half the end-state figure.
- Monthly actives assumed at 6–10% of cumulative installs — deliberately low, reflecting notes-app retention.
- Ad exposure: 15 sessions per active per month × 4 banner impressions per session, at \$0.35–\$0.70 eCPM for mixed-geography productivity traffic.
- Costs: \$25 one-time developer fee, \$12/yr domain, \$0 hosting on a free tier. Your own time is not costed.
- Ad revenue is shown at its theoretical value; section 5 explains why it is not straightforwardly buildable in a TWA.

Sources

1. RevenueCat — *State of Subscription Apps 2026*: median ~\$72/month at 12 months; top 10% \$2,500+; 4.6% reach \$10K/month; freemium 2.1% vs hard-paywall 10.7% day-35 conversion. revenuecat.com/state-of-subscription-apps
2. Adapty — *Productivity app subscription benchmarks 2026*: 14% one-year retention, \$12.99 median monthly price, 29.5% trial-to-paid. adapty.io
3. Business of Apps — 78% of apps released in 2026 have under 1,000 downloads. businessofapps.com
4. Playwire — *AdMob eCPM benchmarks*: banner \$0.50–\$1.50 tier 1, \$0.20–\$0.80 global; productivity apps underperform; plan for 70–80% of calculator estimates. playwire.com
5. MonetizeMore — *How much ad revenue can apps generate in 2026*. monetizemore.com
6. Searchlab — *App marketing & ASO statistics 2026*: 65% of installs via store search; Google Play page-to-install conversion ~26%. searchlab.nl
7. TestFi — Google Play closed testing: 12 unique testers, 14 continuous days, personal accounts created after 13 Nov 2023. testfi.app
8. PrimeTestLab — *Google Play publishing requirements 2026*: \$25 fee, ID verification 1–7 days, API 36 from 31 Aug 2026, 3–5+ week end-to-end timeline. primetestlab.com
9. Android Developers Blog — *Expanded billing choice and lower fees on Google Play* (June 2026): 10% under \$1M, 5% billing fee. android-developers.googleblog.com
10. Play Console Help — *Understanding Google Play's lower service fees*. support.google.com
11. MobiLoud — *Publishing a PWA to the app stores in 2026*: TWA quality gates, Lighthouse ≥ 80, Digital Asset Links, delisting triggers. mobiloud.com
12. GoogleChrome/android-browser-helper issue #535 — no official or supported way to show AdMob banner or rewarded ads within a TWA. github.com
13. Google AdSense Program Policies — Google ads "may not be ... integrated into a software application (does not apply to AdMob) of any kind, including toolbars." support.google.com
14. Chrome for Developers — *Use Play Billing in your Trusted Web Activity* (Digital Goods API + Payment Request API). developer.chrome.com

Prepared 28 July 2026. Benchmark figures are industry medians and ranges, not guarantees; the revenue model is a reasoned estimate, not a forecast. Codebase observations are from `index.html` , `sw.js` and `manifest.json` as of the 28 July 2026 working copy.